

「Weld の Issue (WELD-2834)」で、この挙動が最新の Weld (Jakarta EE 11 向けなど) でどのように改善されようとしているのか？

Weld の Issue (WELD-2834) は、まさに今回議論した「Dependent スコープのインスタンスがメモリを食いつぶす問題」に対する本質的な改善案です。

これまでの Weld は「仕様に忠実であること」を優先し、たとえ不要であっても参照を保持し続けていましたが、最新の Weld (Jakarta EE 11 向けの **Weld 6** 以降や、Jakarta EE 12 を見据えた **Weld 7**) では、  
**\*\*Quarkus ArC に近い「賢い最適化」\*\*が導入されようとしています。**

具体的にどのように改善されるのか、3つのポイントで解説します。

---

## 1. 「破棄の必要性」を事前に判定する

これまでは `get()` で取得した全ての Dependent インスタンスを一律でリストに入れて管理していましたが、改善後は**\*\*「このインスタンスは後でわざわざ壊して (destroy) あげる必要があるか？」\*\***を事前にチェックするようになります。

具体的には、以下の条件を**すべて満たさない**単純な Bean (POJO) であれば、管理リスト (CreationalContext) への追加をスキップします。

- `@PreDestroy` メソッドを持っている
- インターセプターやデコレーターが適用されている
- 独自の Resource インジェクション (`@PersistenceContext` など) を含んでいる

これにより、単純なロジッククラスなどを動的に取得しても、使い終わればすぐに GC の対象になります。

## 2. メモリ参照の「条件付き」スキップ

プルリクエスト (#3314, #3315) での変更点は、`Instance.get()` や `BeanManager.getReference()` を通じてインスタンスを生成する際のロジックの修正です。

内部的なフラグ (`Optimization.isDependentObjectOptimizationAllowed` のような仕組み) を用いて、\*\*「この Bean はライフサイクル終了時に何もする必要がない」\*\*と判断された場合、親の `CreationalContext` にそのインスタンスを登録しない (=ハードリファレンスを持たない) ように挙動が変更されます。

## 3. バージョンごとの対応状況

この修正は、Weld の主要なアクティブバージョンに対して順次取り込まれる方針です。

- **Weld 6.0.5.Final 以降 (Jakarta EE 11 対応版):** このバージョンでこの最適化が導入され、既存の「メモリが減り続ける」という問題が大幅に緩和されます。
- **Weld 7.0.0.Alpha2 以降 (Jakarta EE 12 向け):** 次世代の仕様に向けて、さらにこの「管理しない (Unmanaged)」アプローチが標準化に近い形で組み込まれています。

---

### この改善の影響と注意点

この改善によって、「特別な後処理がいらぬ単純なクラス」については、これまでのメモリリーク問題が自動的に解消されます。ただし、注意が必要なのは以下の点です：

- **互換性の懸念:** 「明示的に `destroy()` を呼んだ時に `@PreDestroy` が動くはず」と期待しているコードがある場合、もし最適化で管理対象から外れていると挙動が変わる可能性があります（ただし、そもそも `@PreDestroy` があれば最適化対象外になるので、安全性は考慮されています）。
- **古い環境 (Jakarta EE 8/10 など) :** 古い WildFly や JBoss EAP を使っている場合、この修正が入っていないため、引き続き「使い終わったら `instance.destroy()` を呼ぶ」という実装がベストプラクティスとなります。

## まとめ

Weld もついに\*\*「仕様を文字通り守る (=メモリを食う)」段階から、「意図を汲み取って最適化する (=Quarkus のように振る舞う)」\*\*段階へと進化しています。

これにより、開発者がメモリ管理を過剰に意識しなくても、より安全に CDI の動的ルックアップを使えるようになる、というわけです。Weld のコミッターがこの Issue に言及しているのは、まさにこの「CDI 実装の歴史的な転換点」を意識してのことだと思われます。